

Phase 1D — Full Super Admin (SA) Portal

Phase 1D delivers the complete Super Admin portal for BaseCenter.ai: tenant management, seat & subscription administration, and platform-wide usage analytics. It builds directly on the Phase 1A-1C foundations (JWT auth, `AdminShell` layout with auth guard, existing admin API/services, and the Stripe integration).

1. Tenant Management

- **List** all tenants with **search** (name / subdomain) and **status filter** (active / suspended).
- **Detail** page per tenant showing owner, status, created date, seat usage, users, and per-module subscription state.
- **Create** a tenant (optionally provisioning an owner user in the same step).
- **Edit** name / subdomain.
- **Suspend / Reactivate** a tenant (keeps `is_active` and `status` in sync).

2. Seat & Subscription Administration

- Per tenant: view **seats allocated vs. used** (used = number of member users), with a visual usage bar, and adjust the allocation.
- Per module: view subscription **status** (active / canceled / past_due / not-enabled) and **seats**, and **enable / disable** modules for the tenant.
- Subscription rows mirror Stripe subscription status where present (`stripe_subscription_id`, status), but every operation works **without Stripe configured** — enable/disable manage local subscription rows directly.

3. Usage Analytics (SA Dashboard)

- Summary cards: **total tenants** (active / suspended breakdown), **users / seats used** (of allocated), **active subscriptions**, and an **estimated MRR** (financial yellow highlight card).
 - **Module adoption** horizontal bar chart: tenants subscribed per module and the MRR each module contributes.
 - MRR is derived from active module subscriptions: `sum(module.monthly_price × seats)` across active/trialing/past_due subscriptions.
-

Backend Changes

Models (`backend/app/models/`)

- `tenant.py` : added `status` (active|suspended), `seats_allocated` , `owner_id` (FK → users) + `owner` relationship. `users / owner` relationships use explicit `foreign_keys` to disambiguate the two FKs to `users` .
- `user.py` : `tenant` relationship pinned to `tenant_id` FK.
- `subscription.py` : added `seats` (quantity per module).

Migration

- `b879cb709462_phaseId_tenant_status_seats_owner_subscription_seats.py` (down-revision `ca8f070d09de`). Adds the new columns with server defaults so it applies cleanly to existing rows, plus the `owner_id` foreign key.

Schemas (`backend/app/schemas/tenant.py`)

- `TenantCreate` , `TenantUpdate` , `TenantSummary` , `TenantDetail` , `TenantModuleSubscription` , `SeatUpdate` , `ModuleToggle` , `MessageResponse` , `AnalyticsOverview` , `ModuleAdoption` .

Services

- `tenant_service.py` : list/get/create/update tenants, suspend/reactivate, set seats, enable/disable modules; summary helpers (seats used, active module count).
- `analytics_service.py` : platform overview metrics + module adoption + MRR.

API (backend/app/api/v1/admin.py) — all protected by get_current_superuser

Method	Path	Purpose
GET	/admin/tenants	list (search, status filters)
POST	/admin/tenants	create tenant (+ optional owner)
GET	/admin/tenants/{id}	tenant detail
PUT	/admin/tenants/{id}	update name/subdomain/seats/owner
POST	/admin/tenants/{id}/suspend	suspend
POST	/admin/tenants/{id}/reactivate	reactivate
PUT	/admin/tenants/{id}/seats	update seat allocation
POST	/admin/tenants/{id}/modules/{module_id}/enable	enable module
POST	/admin/tenants/{id}/modules/{module_id}/disable	disable module
GET	/admin/analytics/overview	platform analytics

Frontend Changes (app/)

- components/admin/AdminShell.tsx : nav now has **Dashboard, Tenants, Stripe Settings**; active-state matches nested routes; wider content area.
- app/admin/page.tsx : login now redirects to /admin/dashboard .
- app/admin/dashboard/page.tsx : analytics dashboard (summary cards + module adoption chart), branded palette.
- app/admin/tenants/page.tsx : tenant list with search/filter + create form.
- app/admin/tenants/[id]/page.tsx : tenant detail — edit, seats, suspend/reactivate, per-module enable/disable.
- lib/api.ts : typed client functions and interfaces for all new endpoints.

Branding palette used throughout: Blue #4F7FFF (primary), Violet #7B68EE (secondary / free modules), Yellow #E4F222 (financial / MRR highlight).

Verification

- Backend imports cleanly; alembic upgrade head applies the new migration.

- All new endpoints exercised via curl (create/list/search/filter, enable/disable modules, seat update, suspend, analytics) — MRR math confirmed.
- Frontend `tsc --noEmit` and `next build` pass; all routes compile.
- Pages verified live in-browser: login → dashboard (cards + chart), tenants list, tenant detail with a live module Enable action.

Local Run

```
# PostgreSQL (native service on port 5432)
# Backend
cd backend && alembic upgrade head && python -m app.db.seed
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
# Frontend
cd app && npm install && npm run dev    # http://localhost:3000/admin
```

Super Admin: `admin@basecenter.ai` / `changeme123`